Selenium Wait:

Selenium Wait is a mechanism in Selenium WebDriver that makes it wait until certain conditions are met before executing further commands of the test script. These typically contain a wait for one of the web page elements to either be loaded or interactable. Without waits, test scripts will fail in trying to perform activities with elements that have not appeared yet, or their loading is not complete; this makes tests unreliable and inconsistent.

The waits in Selenium ensure that your test script will wait (*instead of pausing it with thread.sleep()*) until the condition you name is satisfied-for instance, the presence of an element in the DOM or completing a page load. The concept and usage of these waits are pretty important in correctly designing automated tests of good quality.

Types of Selenium Wait Commands

Selenium provides three different types of wait commands that efficiently handle various timing scenarios and enhance the reliability of automated tests as follows:

1. **Implicit Wait**: It sets a default time for WebDriver to wait for an element to appear before throwing an exception, applying to all elements by default. It's used as a global wait for all elements in your test script and does not wait for specific conditions. Once set, it remains accessible and active throughout the session of the driver.

Syntax:

driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));

How It Works

Whenever there is a defined implicit wait, it will automatically wait for the defined time every time it attempts to locate an element. If it shows up before the time, then it goes ahead with the script without delay. In case it doesn't show up within this given time, Selenium stops the script and throws an exception indicating that the element was not found.

When to Use Implicit Wait

Implicit Wait is extremely useful in cases where you are testing a pretty simple web application because most of the elements load within a predictable time range. While it can help minimize failed test scripts due to small loading delays,

over-reliance on Implicit Wait may lead to longer test execution times if used improperly.

However, it is critical to notice that since the setting of Implicit Wait is global, in complex cases where different elements may require different wait times, it may not be perfect. In such cases, Explicit Wait or Fluent Wait, which offer greater control over specific conditions and timeouts, are more appropriate.

2. **Explicit Wait**: This wait is more targeted oriented, you can set it for what condition you want to continue with the execution. In explicit waits, you can set on an element or on conditions, for instance, till an element is visible or it's clickable.

Syntax:

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));

wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("element_id")
));
```

When to Use Explicit Wait

Explicit Wait is ideal for scenarios where fine-grained control over waiting on a particular element or condition is necessary, and you can even adjust the polling interval for checking conditions. In particular, it is very useful when:

- Dynamic Content: Pages which load elements dynamically, usually driven by technologies like AJAX.
- Specialized Conditions: When you have to wait for specific states of elements, for instance, visibility, clickability, and text presence.
- Unpredictable Load Times: In applications where the load times of elements can't be predicted or vary a lot.

3. **Fluent Wait**: This is an advanced version of the explicit wait. Fluent Wait enables you to define the maximum amount of time to wait for a condition, as well as the frequency with which the WebDriver checks whether the condition is met. It also allows ignoring specific exceptions while waiting, such as NoSuchElementException.